

Teach and Grow: An Agent-Centered Architecture for General Robot Learning

Chang Nie, Zhe Liu, and Hesheng Wang

Abstract—Vision-language-action (VLA) and world-action models often absorb unfamiliar manipulation tasks through additional robot data collection and policy optimization. This recurring retraining burden slows the acquisition of new behavior. We present Teach-and-Grow Learning (TGL), a training-free architecture that turns a few successful demonstrations into reusable robot skills. Task acquisition requires no gradient updates, fine-tuning, or reinforcement learning: pretrained model weights remain fixed as the robot expands its explicit knowledge. Our implementation uses OpenAI GPT-6 Astra for multimodal reasoning and Codex to connect the agent to robot tools. The agent identifies shared subgoals across demonstrations, expresses them as closed-loop Skill Blocks, and grounds each block in the current scene. Physical feedback guides subsequent actions and recovery. Verified behaviors enter a persistent Skill Library, while Experience Memory records the conditions and repairs that inform later decisions. TGL achieves 99.9% mean success across four LIBERO suites and 92.4% across seven LIBERO-Plus perturbation categories. Controlled studies show that taught blocks persist and improve related-task execution with the same model weights and executors. We further formulate a scaling hypothesis relating effective reusable experience to declining future-task error and teaching demand. TGL makes sparse teaching a direct route to lasting robot capability. Code and videos: <https://tgl.changnie.top>.

Index Terms—Training-free robot learning, agentic robotics, GPT-6 Astra, Codex, few-shot teaching, vision-language-action models, lifelong learning, skill composition, embodied intelligence.

I. INTRODUCTION

End-to-end robot foundation models pursue generality through a shared policy that maps observations and language to actions. RT-1 and RT-2 showed how large-scale learning can extend robot behavior [1], [2]; later generalist policies and world-action models expanded the range of tasks and physical dynamics they address [3], [4], [5], [6]. This approach places a substantial burden on training data: the policy must learn to act across the physical conditions it will encounter.

Language models inherited vast collections of text and code produced for other purposes. Robot learning requires an additional kind of evidence: observations paired with actions and their physical consequences on a particular body. New tasks, sensors, and grippers often require collecting this evidence through robot or simulator interaction, followed by policy optimization and regression testing. Web-scale pretraining helps a robot interpret an instruction or recognize an object;

Chang Nie, Zhe Liu, and Hesheng Wang are with the School of Automation and Intelligent Sensing, Shanghai Jiao Tong University, and the Shanghai Key Laboratory of Navigation and Location Based Services, Shanghai 200240, China. Chang Nie: changniep@gmail.com. Corresponding author: Hesheng Wang (e-mail: wanghesheng@sjtu.edu.cn).

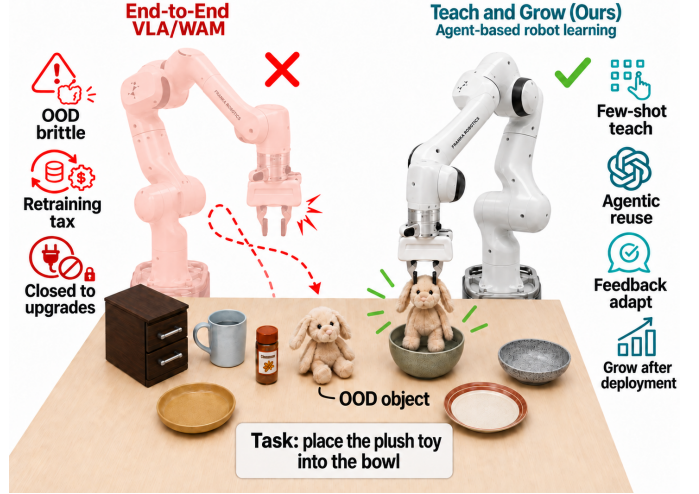


Fig. 1. Two ways to acquire an unfamiliar pick-and-place behavior. *Left*: the illustrated VLA/WAM route incorporates the new plush-toy interaction through a policy-training cycle. *Right*: TGL uses sparse teaching to form reusable Skill Blocks without policy training, composes them through an AI agent, and checks physical outcomes before retaining the behavior for later tasks.

reliable execution also requires geometry, dynamics, and contact information [7], [8], [9], [10], [11].

A failure may concern one object or scene, yet repairing it through a policy update can require regression testing across previously supported behavior. We call this recurring burden the *retraining tax*. As the range of tasks, sensors, and embodiments grows, each update must account for a larger body of existing competence. TGL makes this alternative explicit: a training-free robot acquires new behavior by constructing and validating reusable skills while keeping its pretrained models fixed. New tasks require no gradient update, task-specific fine-tuning, or reinforcement-learning stage.

Coding agents offer a practical starting point. Codex and Claude Code inspect a workspace, form a plan, call tools, and revise their strategy after reading the results [12], [13]. We bring this pattern of interaction to robot manipulation. The agent reads the task and camera observations, chooses skills and tools, inspects the resulting physical changes, and decides what the robot should do next.

Connecting the agent to a robot requires tools for metric geometry, collision checking, contact handling, and continuous control. These tools turn task-level decisions into motion and return evidence of what happened. After each meaningful action, the agent checks that evidence before choosing the next step.

TGL · TEACH-AND-GROW LEARNING · 教学相长

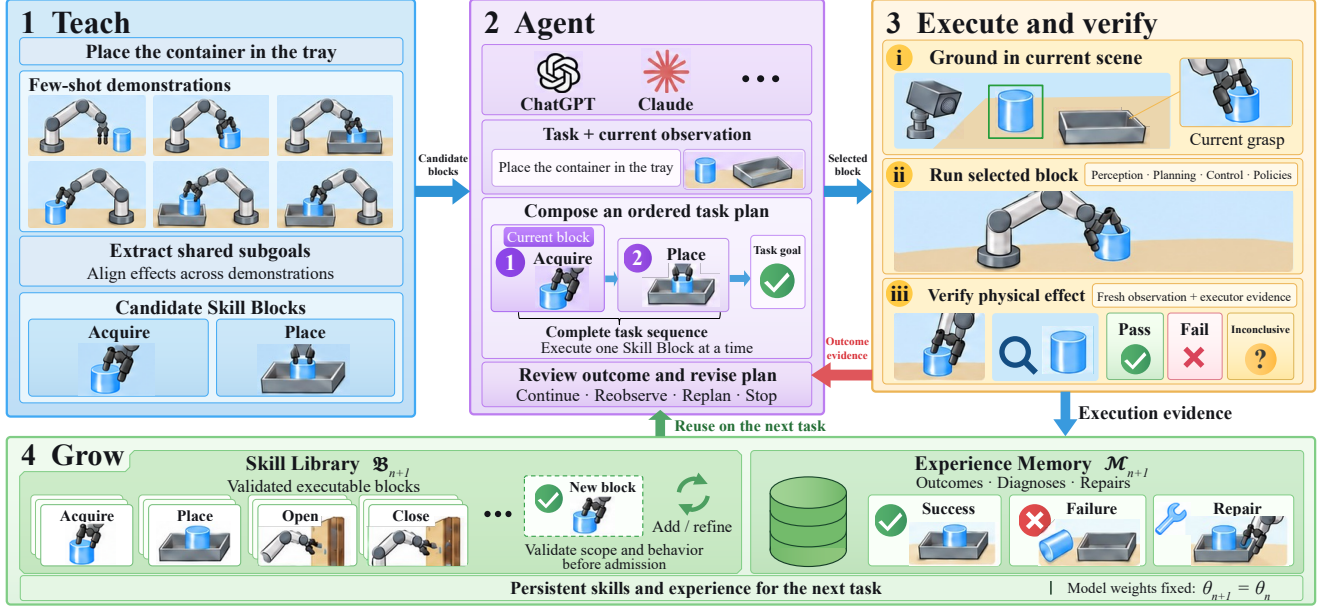


Fig. 2. Teach-and-Grow for one unfamiliar task. Teaching reveals shared state changes and new Skill Blocks. The AI Agent composes a route, runs one block, and checks its effect after the robot acts. Verified capabilities enter the Skill Library; outcomes, failures, and repairs enter Experience Memory. The complete Grow state returns to composition on the next task.

ASPIRE and related systems show that zero-shot agent exploration can assemble and repair robot behavior without updating a task-specific policy [14], [15]. On a physical robot, however, learning mainly through trial requires many sequential decisions and can trigger unsafe exploratory actions. Teach-and-Grow therefore uses sparse successful teaching to seed task structure and reserves exploration for the remaining gaps.

We propose Teach-and-Grow Learning (TGL), an architecture for training-free skill acquisition from sparse teaching. A few successful demonstrations reveal the task’s semantic subgoals and their ordering. By analyzing the demonstrations together, the agent extracts a shared strategy and represents it as reusable Skill Blocks. Each block preserves the intended effect and the strategy for achieving it, while its objects, geometry, and control targets are computed from the current scene. The learned behavior can therefore adapt to conditions that differ from the original trajectory. Our implementation uses OpenAI GPT-6 Astra [16] for multimodal reasoning and Codex for interaction with robot tools. Training-free refers to acquiring the incoming task with fixed pretrained weights; demonstrations update executable skills and memory directly.

Teach-and-Grow draws on the classical Chinese principle 教学相长 (*jiào xué xiāng zhǎng*), translated here as *teaching and learning promote one another*. Demonstrations give the robot an initial task structure. Its attempts reveal missing knowledge and help the teacher choose the next intervention. As reusable skills accumulate, teaching can focus on unfamiliar behavior and unresolved failures.

At task time, the agent retrieves and composes blocks, gathers the evidence each block needs, and executes one meaningful stage before looking again. The physical result

determines what follows. The agent may continue, repair the remaining composition, or explore a missing piece; it can also request targeted teaching when another demonstration would introduce a new behavior or shorten acquisition. Teaching gives exploration a productive starting point, and exploration lets the system grow beyond the teacher’s demonstrations.

After execution, validated behaviors enter a persistent *Skill Library*. A structured textual *Experience Memory* records the conditions under which they succeeded or failed and the repairs associated with recovery. These stores provide an editable account of executable skills and the experience that guides their use. New behavior can be added locally, with earlier skills remaining separately available for future composition.

The Agent decides which subgoal to pursue, what evidence is needed, and whether the last action achieved its intended effect. Robot-native executors handle metric motion, contact, and continuous control. Physical evidence connects these two levels: the executor reports an outcome, and the Agent uses it to revise the task route. This division also makes individual components easier to improve. A grasp detector can be replaced within a block, a recovery can be attached to a failed effect, and a validated block can enter the library as a separately retrievable behavior.

Fig. 1 illustrates how this changes task acquisition. The end-to-end route incorporates new interaction through a policy update; TGL turns teaching into explicit behavior that can be grounded, tested, and reused. Learned policies, planners, perception models, and controllers supply the physical capabilities. The Agent selects and coordinates them according to the task and the observed state.

TGL separates the acquisition of unfamiliar behavior from

the fast execution of familiar behavior. The Agent inspects outcomes, changes the remaining plan, and retains validated skills. Once a behavior is stable, a VLA, WAM, or other fast policy can execute its block or, in a future distillation stage, learn from its verified trajectories. A new task can therefore become executable as soon as its taught skills pass validation, without waiting for a policy-training cycle.

This lifecycle also changes the quantity that should scale. Neural scaling laws relate loss to resources such as parameters, data, and compute [17], [18]. Let $X > 0$ denote the robot’s *effective reusable experience*: accumulated interaction that has survived validation and can still be retrieved, grounded, and composed in a new scene. We propose the Teach-and-Grow scaling-law hypothesis

$$\begin{aligned}\mathcal{E}_{\text{future}}(X) &= \mathcal{E}_{\infty} + AX^{-\alpha}, \\ D_{\text{teach}}(X) &= D_{\infty} + BX^{-\beta},\end{aligned}\quad (1)$$

where $\mathcal{E}_{\text{future}}$ is expected error on related future tasks, D_{teach} is the teaching needed to acquire one, $\mathcal{E}_{\infty}, D_{\infty} \geq 0$, and $A, B, \alpha, \beta > 0$. Unlike model size or a frozen offline dataset, X can grow throughout the robot’s life. The hypothesis predicts that retaining more reusable experience reduces the teaching needed for related new tasks.

Our contributions are:

- We formulate **Teach-and-Grow Learning**, a training-free architecture that acquires manipulation skills from sparse teaching and reaches 99.9% mean success on LIBERO and 92.4% on LIBERO-Plus with fixed pretrained weights.
- We introduce reusable closed-loop **Skill Blocks** that turn several demonstrations into shared subgoal strategies while recomputing physical realization from the current scene.
- We couple a persistent **Skill Library** with structured **Experience Memory**, allowing successful behavior, failure knowledge, and repair to shape later tasks and future fast-policy students.
- We propose a **Teach-and-Grow scaling-law hypothesis** for lifelong robot intelligence, relating effective reusable experience to future-task error and the marginal amount of teaching required for new tasks.

II. RELATED WORK

A. Generalist VLA and World-Action Policies

VLA models learn a direct mapping from visual-language observations to robot actions. RT-1 and RT-2 established the promise of large-scale behavior cloning and web-to-robot transfer [1], [2]. Subsequent generalist policies broadened the route across model families [3], [4], [5], while larger robot datasets expanded its empirical foundation [7], [8], [9]. Across these efforts, scaling data and model resources produces systematic gains [19], [20], [21].

These results support end-to-end scaling, but they also expose its binding resource: physical coverage. Semantic knowledge from web-scale pretraining does not by itself supply action-conditioned geometry, contact, embodiment dynamics, or recovery. Probing studies report a gap between instruction understanding and reliable execution under distribution shift [11]. Online reinforcement learning and few-shot policy

modules reduce the adaptation burden [22], [23], [24], while world-action models such as DreamZero learn visual dynamics and actions jointly [6]. The remaining systems question is how a robot should absorb uncovered tasks, modalities, and failures when another global training cycle is too costly or too slow.

TGL organizes these policies within a broader learning system. A generalist policy can execute a block, while the Agent acquires and validates new task structure through teaching and interaction. The resulting explicit behaviors can later supply trajectories for fast-policy distillation.

B. Demonstrations, Few-Shot Adaptation, and Reusable Skills

Learning from demonstration supplies successful structure without unrestricted physical exploration. Existing systems use demonstrations, play, and language to generate behavior that can be transferred or recombined [25], [26], [27]. A closely related line makes skill structure explicit for continual learning, symbolic composition, or few-shot adaptation [28], [29], [23].

In TGL, demonstrations support the inference of semantic subgoals, reusable relations, ordering, expected effects, and applicability. A Skill Block can contain several perception-action iterations and use a learned policy, planner, servo, controller, or tool composition. Its strategy is grounded in current observations at execution time, allowing the same subgoal to be realized through different motions.

C. Agentic Tool Use, Memory, and Lifelong Robots

Language-conditioned planners and tool-using agents have shown that high-level reasoning can organize robot programs, constraints, and feedback [30], [31], [32]. Recent robot operating layers extend this idea to skill graphs, memory, and cross-embodiment interfaces [33], [34], [35]. Other systems retain deployment experience or grow and repair skill libraries over time [36], [37], [14].

Episodic memory, tools, and executable skills play distinct roles. Memory records what happened and why an attempt failed. A tool provides an operation the robot can invoke. An executable skill connects an intended change to current-scene grounding, an appropriate executor, and an outcome test. Storing more episodes therefore increases the available context, while adding a validated skill expands the behaviors available for composition. TGL uses memory to guide selection and Skill Blocks to represent what the robot can execute.

TGL connects these ideas through executable skills acquired without task-specific training. Demonstrations give the agent a subgoal strategy that it can test in the current scene and retain after validation. During execution, observations determine whether that strategy should continue or change. The resulting block and its associated experience then guide subsequent tasks, so each teaching interaction can contribute behavior beyond its original episode.

III. TEACH-AND-GROW LEARNING

A. Learning with Fixed Model Weights

Training-free task acquisition in TGL expands explicit skills and memory while keeping the pretrained agent and robot

models fixed. Demonstrations serve as reasoning inputs, and no optimizer is invoked for the new task. Consider a sequence of tasks $\{\mathcal{T}_n\}_{n=1}^N$. When task $\mathcal{T}_n$ arrives, the robot receives a small teaching set

$$\mathcal{D}_n = \{d_n^{(1)}, \dots, d_n^{(m_n)}\}, \quad m_n \ll |\mathcal{D}_{\text{pretrain}}|, \quad (2)$$

where $\mathcal{D}_{\text{pretrain}}$ denotes the corpus used to train the fixed foundation stack. The robot begins with a Skill Library $\mathcal{B}_n$, Experience Memory $\mathcal{M}_n$, and interaction history $\mathcal{H}_n$. Let $\Delta\mathcal{H}_n$ contain the useful lessons from executing this task. The update is

$$\begin{aligned} \theta_{n+1} &= \theta_n = \theta, \\ (\mathcal{B}_{n+1}, \mathcal{M}_{n+1}) &= \mathcal{U}(\mathcal{B}_n, \mathcal{M}_n, \mathcal{D}_n, \Delta\mathcal{H}_n), \\ \mathcal{H}_{n+1} &= \mathcal{H}_n \cup \Delta\mathcal{H}_n. \end{aligned} \quad (3)$$

New behaviors enter $\mathcal{B}$, and their context, failures, and repairs enter $\mathcal{M}$. With θ fixed, learning changes the behaviors the robot can retrieve and execute.

Equation 3 assigns a separate role to each part of the persistent state. The fixed parameters θ provide pretrained semantic and physical priors. The Skill Library $\mathcal{B}$ changes when a block passes its admission checks. Experience Memory $\mathcal{M}$ records task context, outcomes, diagnoses, and repairs to guide later decisions. The history $\mathcal{H}$ preserves the underlying evidence, and X summarizes the part that remains verified and reusable. A future fast student has separate parameters ϕ and can be trained on verified trajectories. Thus, task acquisition updates explicit skills and memory with θ fixed; later distillation updates ϕ to compress the acquired behavior.

Behavior cloning turns demonstrations into supervised action targets, while reinforcement learning uses interaction to change a policy or value function. TGL treats demonstrations as evidence about task structure and makes executable capability itself the object of learning.

B. Skill Blocks

A Skill Block is represented as

$$b_i := \langle g_i, \mathcal{S}_i, \rho_i, \gamma_i, \Pi_i, v_i, \mathcal{R}_i \rangle. \quad (4)$$

Each field answers one practical question: g_i says what should change; $\mathcal{S}_i$ says when the block applies; ρ_i stores the reusable strategy; $\gamma_i : \mathcal{O} \rightarrow \mathcal{Z}_i$ maps the observation space $\mathcal{O}$ to the block's grounded variables $\mathcal{Z}_i$; Π_i lists compatible executors; v_i maps pre-action observations, post-action observations, and executor evidence to *pass*, *fail*, or *inconclusive*; and $\mathcal{R}_i$ lists bounded recovery choices. At decision step t , the Agent selects an executor $\pi_{i,t} \in \Pi_i$.

The word *block* emphasizes composition rather than a low-level motor primitive. A block for acquiring the requested container turns a semantic request into a grounded grasp and closes its loop by verifying retention. The next block only needs the achieved effect: the target object is held. It does not depend on the backend that produced that effect.

A complete task is represented at each semantic decision step by an ordered working plan

$$\tau_t = (b_{i_{t,1}}, b_{i_{t,2}}, \dots, b_{i_{t,L_t}}), \quad (5)$$

where L_t is the current route length. After a block returns an outcome, the agent may keep, shorten, or replace the remaining route. The tuple represents the remaining plan at the current decision step.

C. Structured Experience Memory

A Skill Block stores a behavior; Experience Memory stores the lesson learned from using it. Each record is a structured textual item

$$\mu_j = \langle \text{task, context, blocks, outcome,} \\ \text{diagnosis, repair, evidence} \rangle. \quad (6)$$

Each record describes the outcome, its diagnosis, and any successful repair. It retains links to observations and provenance and is retrieved as compact guidance that a person can inspect or revise. This gives later runs access to the conditions behind a skill's success or failure.

D. The Teach-and-Grow Scaling-Law Hypothesis

The scaling resource in TGL measures how much past experience remains useful for action in a new scene. At a checkpoint with at least one positively weighted experience, define

$$X_n = \sum_{h \in \mathcal{H}_n} \omega(h; \mathcal{B}_n, \mathcal{M}_n), \quad (7)$$

where $\omega(h; \mathcal{B}_n, \mathcal{M}_n) \in [0, 1]$ is a dimensionless score fixed before evaluation; the positive-weight condition above ensures $X_n > 0$. The score uses only evidence available at checkpoint n : evidence reliability, added coverage, retrievability, grounding validity, and compatibility with admitted blocks. Future-task outcomes never enter it. Thus, two robots can store the same number of episodes yet possess very different X_n . In Eq. 1, X denotes this same checkpoint quantity under the fixed scoring rule.

Equation 1 states the central prediction. As effective experience grows, error on related future tasks and the teaching required to acquire them should fall toward task-dependent floors. The exponents α and β characterize how efficiently a system turns lived experience into future competence. A robot that records many episodes but cannot retrieve or re-ground them will scale poorly; one that turns a small number of verified episodes into broadly reusable blocks may scale well.

The law can be tested over sequential acquisition experiments: hold the foundation models and tools fixed, grow X , and measure future-task error and marginal teaching at each checkpoint. We define D_{teach} as teacher intervention time under a fixed protocol until the preregistered success criterion is reached. If the criterion is not reached before the preregistered teaching budget is exhausted, the observation is recorded at that cap rather than omitted. Because Skill Blocks remain explicit objects, the same study can also track whether new growth preserves earlier behavior.

IV. GENERAL AGENTIC ROBOT ARCHITECTURE

Fig. 2 follows one unfamiliar task from teaching to reuse. Demonstrations provide its subgoal structure. The agent selects blocks and robot tools, checks the result of each action, and retains useful behavior and experience for later tasks. The architecture defines how the agent, reusable blocks, and executors exchange decisions and physical evidence. The choice of reasoning model, agent runtime, and robot tools is left to the implementation.

The Agent interprets the task, retrieves candidate blocks, and decides what remains to be done. Each selected block identifies the relevant objects and geometry in current observations before its executor acts. Fresh observations and executor feedback then determine whether the intended effect occurred. Before a block enters the library, validation checks its supported scope, executor compatibility, outcome test, recovery behavior, and provenance. The same contract supports later improvements: a detector, planner, controller, or learned policy can be replaced and tested within the block.

A. Few-Shot Teaching and Cross-Demonstration Abstraction

For a new task, a few successful demonstrations show the state changes needed to reach the goal. The learner identifies events such as acquiring an object, opening a drawer, or placing one object inside another. These events define candidate subgoals for the new task.

The AI agent reads the instruction together with synchronized observations. Each demonstration is then viewed as an ordered sequence of segments, divided around meaningful changes in task state:

$$d^{(j)} = (s_1^{(j)}, s_2^{(j)}, \dots, s_{K_j}^{(j)}). \quad (8)$$

Segments are aligned by what they accomplish, even when their timing and motion differ. Let $\mathcal{A}_k$ collect all segments aligned to semantic effect k . The agent then synthesizes the shared strategy

$$\rho_k = \text{Synthesize}(\mathcal{A}_k). \quad (9)$$

The agent keeps the shared roles, relations, ordering, and effects, then recomputes scene-specific details such as pose, grasp, path, and control. Differences among demonstrations show how widely the block can be reused. A pattern seen with one instrument stays narrow; a strategy repeated across interchangeable objects can support a broader scope.

B. Skill Blocks in the Current Scene

Fig. 3 shows how a block combines a reusable strategy with current observations. Its goal and strategy persist across tasks; object poses, grasp candidates, and motions are computed for the scene in front of the robot.

A new block is evaluated in scenes separate from its teaching demonstrations. Successful executions define the supported range; failures guide refinement or narrower scope. The block enters the library after validation.

Different backends may realize the same block as long as they achieve and verify the same result. An acquisition block may use geometric grasping, a VLA, or a tactile policy. Adding

a new sensor or executor therefore changes only the blocks that need it.

C. Dynamic Agentic Composition and Tool Selection

Given a task instruction, current observations, and persistent state $(\mathcal{B}_n, \mathcal{M}_n)$, the Agent retrieves a compact set of blocks and forms the working plan in Eq. 5. It selects executors according to the current evidence. A familiar scene may use a learned policy, while uncertain contact may call for geometry, touch, or a more deliberate controller. The Agent can revise this choice at each semantic state transition.

Continuous motor control runs within the selected executor. At semantic decision step t , a block returns the next observation and outcome evidence:

$$(o_{t+1}, \varepsilon_t) = \text{Run}(b_{i_{t,1}}, \pi_{i_{t,1},t}, o_t). \quad (10)$$

Here ε_t records executor and verifier evidence, including whether the intended effect passed, failed, or remained inconclusive. The agent uses it to continue, look again, choose another route, ask for teaching, or stop. What remains to be done depends on what physically happened.

Task-level decisions can change both the remaining subgoals and the tools used to reach them. The Agent may request another sensor observation, call a geometric planner, retrieve a different behavior, or replace part of the block sequence. The executor continues to handle its local action loop and returns evidence at semantic boundaries. This allows continuous control and task-level replanning to operate at their appropriate timescales.

D. Physical Feedback, Bounded Exploration, and Targeted Teaching

After a block executes, the agent compares the observed effect with the intended change. If they agree, the task continues. If they differ, the agent looks again, selects another executor or route, or asks the teacher for a focused demonstration. Once a mismatch is detected, the plan is corrected before the next block runs.

Targeted teaching broadens this loop. The agent can ask for another demonstration when it would introduce a missing behavior, clarify an ambiguous subgoal, or move acquisition forward faster than unguided trial. Teaching seeds productive exploration; interaction broadens the teaching set; later guidance becomes increasingly selective.

E. Structured Growth and Fast-Policy Distillation

Execution contributes to two persistent stores. Validated reusable behavior enters the Skill Library; Experience Memory records outcomes, diagnoses, and repairs. Both stores are searchable, versioned, and human-editable, so later tasks can reuse a behavior together with the context that supports its selection.

Verified trajectories can also train a fast student. A VLA or WAM may execute a mature block directly, while the agentic path continues to handle novelty, diagnosis, and capability expansion. As robot foundation models become stronger,

SKILL BLOCK · REUSABLE CONTRACT, CURRENT-SCENE EXECUTION

$$b_i := \langle g_i, S_i, \rho_i, \gamma_i, \Pi_i, v_i, \mathcal{R}_i \rangle$$

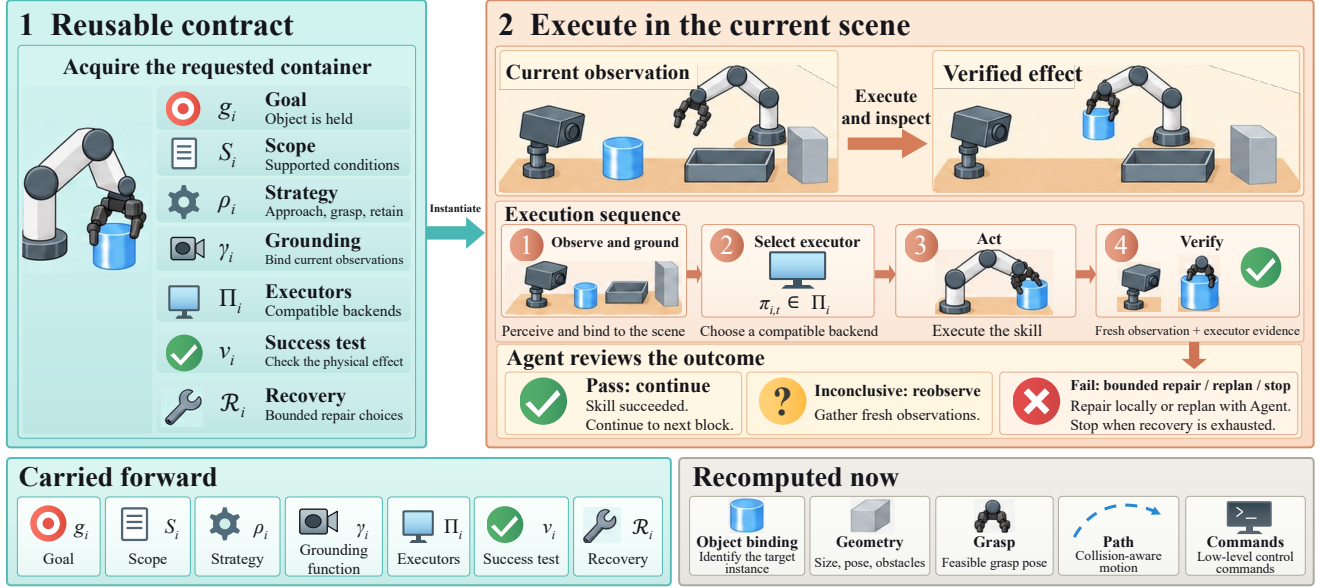


Fig. 3. A Skill Block is a reusable contract for one meaningful change. It carries a goal, scope, strategy, grounding function, executor set, success test, and recovery. At runtime it observes the scene, binds current objects and geometry, chooses an executor, acts, and verifies the result.

several specialist tools may collapse into a single executor. The agent continues to acquire unfamiliar behavior and verify its use, while each executor supplies the motion needed for the current subgoal.

V. EXPERIMENTS AND RESULTS

We evaluate TGL on LIBERO [38] and LIBERO-Plus [39]. LIBERO-Plus extends the original benchmark with perturbations to object layouts, camera viewpoints, robot initial states, instructions, lighting, backgrounds, and sensor observations. The benchmark evaluations assess task success, while controlled studies examine how demonstrations become executable blocks, how those blocks persist, and how feedback changes the plan. Appendix I gives the controlled-study protocols and trace provenance.

Tables I and II summarize success rates on LIBERO and LIBERO-Plus.

On LIBERO, TGL reaches 99.9% mean success, matching LaST-R1 for the highest mean among the compared methods. It exceeds OpenVLA-OFT by 2.8 percentage points and SRPO by 0.7 points, while acquiring its skills without policy optimization. Object and Goal reach 100.0%, and Long reaches 99.9%.

On LIBERO-Plus, TGL obtains the highest category mean, 92.4%, compared with 89.7% for $\pi_{0.5}$ with RAS and MCSI, 85.8% for InternVLA-A1.5, and 84.5% for VLANeXt. TGL leads on robot initial states, language, lighting, and layout. Its camera and sensor-noise scores identify the perturbations for which stronger visual grounding would be most useful. The controlled studies below examine how taught skills and physical feedback contribute to execution.

a) From demonstrations to persistent blocks.: Ten visual demonstrations decompose into acquisition and release stages,

and all 20 observable effects are confirmed. In a separate learning cycle, three teacher trajectories produce two blocks: acquire the requested object and release it in the required relation. Together they solve all three evaluation states, are saved, reloaded, and solve the same states again. Save-and-reload shows that the taught route has joined the library as a reusable alternative; the existing six-block route remains available on the same states. Beyond its learned range, the route stops when the expected effect is absent, preventing one mistake from propagating through the task. The reported visual study uses a deterministic decomposition, while the general formulation can place a multimodal Agent in this role.

b) Feedback changes the workflow.: For the bowl-on-plate task, the Agent begins with an acquisition block followed by a release block. The pick succeeds, but the post-grasp evidence is not yet sufficient for placement. The Agent rebuilds the remaining route and completes the task. For the drawer-opening task, an inconclusive outcome prompts a fresh observation. In both traces, evidence from the physical world changes the Agent’s next action.

c) Local growth under a fixed executor.: The six-block library solves 0/6 evaluations; adding the two learned blocks raises success to 4/6. Model weights, runtime, and evaluation budget remain fixed. This two-task pilot illustrates how library content can change the behavior available to the same executor. The corresponding uncertainty estimates and acquisition costs are reported in Appendix I.

VI. DISCUSSION

A. Learning as Capability Growth

Most learning systems store new ability inside a policy, value function, or world model. TGL also stores it as reusable

TABLE I

LIBERO SUCCESS RATES (%). MEAN AVERAGES THE FOUR SUITES EQUALLY. BOLD MARKS THE BEST RESULT IN EACH COLUMN, INCLUDING TIES.

Method	Spatial	Object	Goal	Long	Mean
OpenVLA [40]	84.7	88.4	79.2	53.7	76.5
OpenVLA-OFT [40]	97.6	98.4	97.9	94.5	97.1
SRPO [41]	98.8	100.0	99.4	98.6	99.2
VLANeXt [42]	99.0	99.2	96.6	94.8	97.4
InternVLA-A1.5 [43]	98.6	99.8	98.6	98.4	98.9
LaST-R1 [44]	99.8	100.0	100.0	99.8	99.9
TGL (ours)	99.7	100.0	100.0	99.9	99.9

TABLE II

LIBERO-PLUS SUCCESS RATES (%). MEAN AVERAGES THE SEVEN PERTURBATION CATEGORIES EQUALLY. BOLD MARKS THE BEST RESULT IN EACH COLUMN.

Method	Camera	Robot	Language	Light	Background	Noise	Layout	Mean
OpenVLA [39]	0.8	3.5	23.0	8.1	34.8	15.2	28.5	16.3
π_0 [39]	13.8	6.0	58.8	85.0	81.4	79.0	68.9	56.1
OpenVLA-OFT [39]	56.4	31.9	79.5	88.7	93.3	75.8	74.2	71.4
OpenVLA-OFT+ [39]	92.8	30.3	85.8	94.9	93.9	89.3	77.6	80.7
VLANeXt [42]	90.4	65.7	81.8	95.9	82.5	94.1	80.8	84.5
InternVLA-A1.5 [43]	83.1	55.1	86.9	96.4	98.2	95.6	85.2	85.8
$\pi_{0.5}$ + MCSI [45]	86.0	83.0	83.0	97.0	98.0	92.0	87.0	89.4
$\pi_{0.5}$ + RAS + MCSI [45]	81.0	88.0	90.0	97.0	96.0	90.0	86.0	89.7
TGL (ours)	87.3	90.7	96.8	97.3	97.4	89.9	87.2	92.4

TABLE III

CONTROLLED TEACH-AND-GROW STUDIES ON LIBERO. DETAILED SPLITS AND STATISTICS APPEAR IN APPENDIX I.

Study	Observed result	Main lesson
Task-specific learning cycle	Two learned blocks solve 3/3 states, survive save-and-reload at 3/3, and stop at the first unmet semantic effect outside their learned scope	Demonstrations become persistent, scope-aware behavior
Feedback-driven execution	Two representative successful traces replan or reobserve after physical feedback	Outcomes change what the Agent does next
Fixed-executor library pilot	Six-block library: 0/6; eight-block library: 4/6	Local library growth changes what the same executor can do

behavior and the experience that gives that behavior meaning. A Skill Block may be realized by a learned policy, planner, controller, or conventional program. It becomes learned knowledge when the robot can recognize where it applies and produce the intended result in a new scene.

Few-shot teaching supplies the subgoal order and success conditions. The robot then computes geometry and control from the current scene. This division narrows exploration while allowing execution to reveal variations and repairs beyond the demonstrated trajectories.

B. Persistent Memory and Non-Destructive Growth

The Skill Library and Experience Memory make growth explicit. A new task leaves behind a named behavior and the context that makes it useful. When a related task arrives, the Agent retrieves that behavior, checks its applicability, and grounds it in the new scene.

Experience records explain why a block was selected, which effect was expected, and what the robot actually observed. This information connects a failure to the decision that produced it. If an acquisition block closes the gripper but the next observation does not confirm object retention, the remaining placement

route lacks its required input. The agent can request another view or revise the route before continuing. A record of that mismatch can guide later block selection and verification. The executable response and the diagnostic record remain separate: a revised behavior enters the library after validation, while its supported conditions and outcome stay available in memory. This separation makes correction local and keeps earlier skills individually addressable as new behavior is added.

C. The Agent as Organizing Intelligence

At each semantic boundary, the Agent checks whether the plan still fits the observed state. It can request new evidence or replace the remaining route while dedicated executors handle motion. The two LIBERO traces show both decisions.

Perception models, geometric planners, servos, controllers, and VLA policies provide the physical capabilities on which this reasoning acts. A stronger executor may combine several of these functions. The Agent can then work with fewer tools while continuing to select subgoals, inspect outcomes, and retain useful behavior.

EXPERIENCE-DRIVEN SCALING

Hypothesis and future fast-policy extension

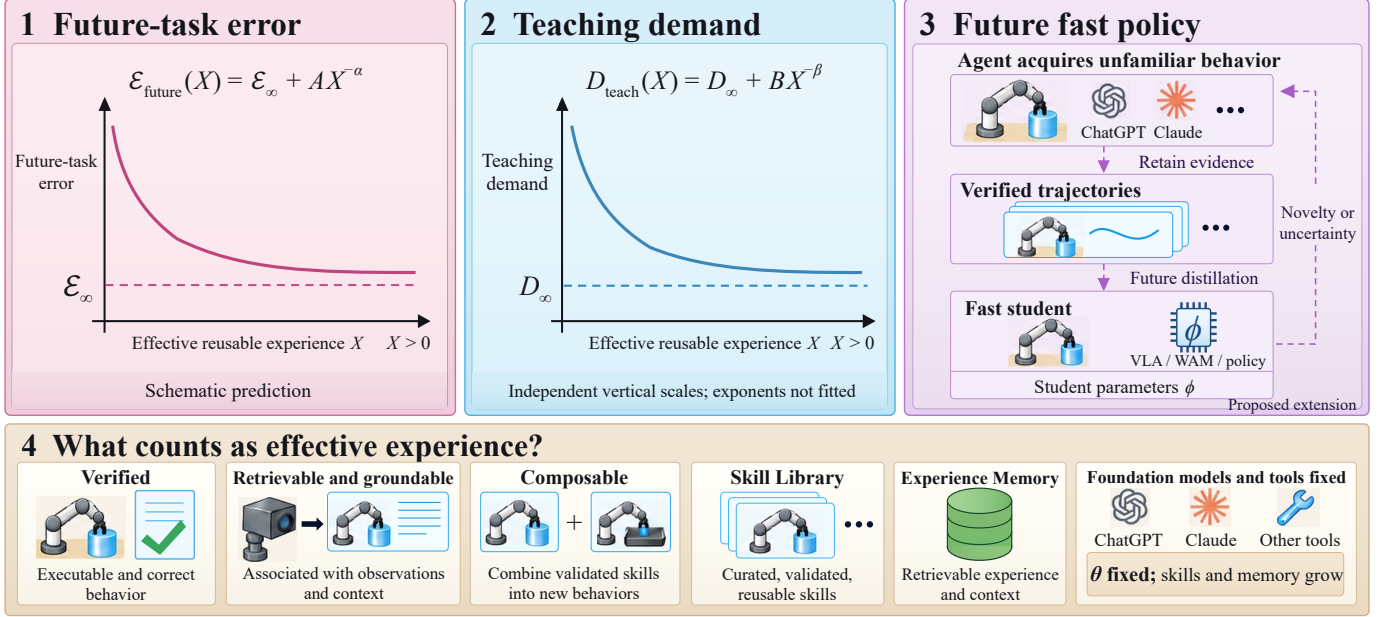


Fig. 4. Experience-driven robot scaling. The two aligned plots use separate schematic vertical scales: as effective reusable experience X grows, future-task error and teaching demand are predicted to approach their own irreducible floors. The Agent learns unfamiliar tasks and keeps verified experience; a future compression path moves mature behavior into a fast policy, with uncertain cases returning to the Agent.

D. The Teach-and-Grow Scaling-Law Hypothesis

Model scaling improves the prior before deployment. Test-time scaling spends more computation on one episode. Fig. 4 adds lifetime scaling: useful experience from one task changes how the next task begins. A single bridge block can open many new compositions; a large archive of redundant trajectories may add almost nothing.

With foundation models and tools fixed, Eqs. 1 and 7 predict declining future-task error and teaching demand as X grows. Appendix G defines checkpoints and cost accounting for testing these predictions.

E. A Slow Teacher and a Fast Student

In the proposed two-speed extension, the agentic route learns unfamiliar tasks while a distilled fast policy executes familiar ones. Verified agentic trajectories can train a VLA, WAM, diffusion policy, or block-specific controller. When the fast path is uncertain, control returns to the Agent, which diagnoses the gap and expands the library.

The same division can later extend to a fleet: robots may share candidate strategies and failure signatures, while each receiver re-grounds and verifies them locally.

F. Execution Cost

Sequential reasoning and repeated observations add latency to unfamiliar tasks. TGL places these decisions at subgoal boundaries, while dedicated executors handle continuous control. Retrieval quality and current-scene grounding determine whether a stored skill remains useful as the library expands.

VII. CONCLUSION

Teach-and-Grow Learning provides a training-free path from sparse demonstrations to reusable robot capability. OpenAI GPT-6 Astra reasons over the task and physical feedback, while Codex connects its decisions to robot tools. New tasks expand the Skill Library and Experience Memory with pretrained model weights fixed. This implementation achieves 99.9% mean success on LIBERO and 92.4% on LIBERO-Plus, matching the highest LIBERO mean and achieving the highest perturbation-category mean among the compared methods.

The controlled studies connect these results to concrete learning operations. Demonstrations produce executable subgoal blocks that survive save-and-reload, and two additional blocks raise related-task success from 0/6 to 4/6 with the same executor. Physical feedback also changes subsequent decisions through reobservation and replanning. Each retained block gives the robot a behavior it can ground again, while the associated experience explains when that behavior is useful. Explicit skills also make correction local. A failed effect identifies the behavior or observation that needs attention, and the revised block is validated before reuse. Earlier skills remain separately addressable, allowing the robot to expand its repertoire without a global policy update.

The Teach-and-Grow scaling hypothesis makes reusable experience a resource for reducing future-task error and teaching demand. Mature behavior can also supply trajectories for a fast student. TGL thus gives deployment a continuing learning role: a robot acquires a skill through teaching, puts it to work with fixed model weights, and adapts the retained behavior to new manipulation tasks.

REFERENCES

- [1] A. Brohan *et al.*, “RT-1: Robotics transformer for real-world control at scale,” 2022. [Online]. Available: <https://arxiv.org/abs/2212.06817>
- [2] B. Zitkovich *et al.*, “RT-2: Vision-language-action models transfer web knowledge to robotic control,” 2023. [Online]. Available: <https://arxiv.org/abs/2307.15818>
- [3] Octo Model Team *et al.*, “Octo: An open-source generalist robot policy,” 2024. [Online]. Available: <https://arxiv.org/abs/2405.12213>
- [4] M. J. Kim *et al.*, “OpenVLA: An open-source vision-language-action model,” 2024. [Online]. Available: <https://arxiv.org/abs/2406.09246>
- [5] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, S. Levine, A. Li-Bell, M. Mollhuthuri, S. Nair, K. Pertsch, L. X. Shi, J. Tanner, Q. Vuong, A. Walling, H. Wang, and U. Zhilinsky, “ π_0 : A vision-language-action flow model for general robot control,” 2024. [Online]. Available: <https://arxiv.org/abs/2410.24164>
- [6] S. Ye, Y. Ge, K. Zheng, S. Gao, S. Yu, G. Kurian, S. Indupuru, Y. L. Tan, C. Zhu, J. Xiang, A. Malik, K. Lee, W. Liang, N. Ranawaka, J. Gu, Y. Xu, G. Wang, F. Hu, A. Narayan, J. Bjorck, J. Wang, G. Kim, D. Niu, R. Zheng, Y. Xie, J. Wu, Q. Wang, R. Julian, D. Xu, Y. Du, Y. Chebotar, S. Reed, J. Kautz, Y. Zhu, L. J. Fan, and J. Jang, “World action models are zero-shot policies,” 2026. [Online]. Available: <https://arxiv.org/abs/2602.15922>
- [7] Open X-Embodiment Collaboration *et al.*, “Open x-embodiment: Robotic learning datasets and RT-X models,” 2023. [Online]. Available: <https://arxiv.org/abs/2310.08864>
- [8] A. Khazatsky *et al.*, “DROID: A large-scale in-the-wild robot manipulation dataset,” 2024. [Online]. Available: <https://arxiv.org/abs/2403.12945>
- [9] K. Wu *et al.*, “RoboMIND: Benchmark on multi-embodiment intelligence normative data for robot manipulation,” 2024. [Online]. Available: <https://arxiv.org/abs/2412.13877>
- [10] AgiBot-World-Contributors *et al.*, “AgiBot World Colosseo: A large-scale manipulation platform for scalable and intelligent embodied systems,” 2025. [Online]. Available: <https://arxiv.org/abs/2503.06669>
- [11] I. Fang, J. Zhang, S. Tong, and C. Feng, “From intention to execution: Probing the generalization boundaries of vision-language-action models,” 2025. [Online]. Available: <https://arxiv.org/abs/2506.09930>
- [12] OpenAI, “Introducing codex,” May 2025, official product description, accessed August 2026. [Online]. Available: <https://openai.com/index/introducing-codex/>
- [13] Anthropic, “Claude code overview,” 2025, official documentation, accessed August 2026. [Online]. Available: <https://docs.anthropic.com/en/docs/claude-code>
- [14] R. Lu, Y. Wu, E. Kou, L. Fu, W. Xiao, A. Mandlekar, Y. Xu, G. Shi, K. Goldberg, A. Chen, M. Chowdhury, Y. Zhu, L. J. Fan, and G. Wang, “ASPIRE: Agentic skills discovery for robotics,” 2026. [Online]. Available: <https://arxiv.org/abs/2607.00272>
- [15] J. Zhang, J. Ge, H. Yoo, L. Fu, Z. Yang, Y. Liu, R. Saravanan, S. Yin, J. Yu, D. Niu, Z. Wang, R. Herzig, K. Goldberg, Y. Bai, D. M. Chan, I. Stoica, A. Kanazawa, J. Lei, H. Feng, and T. Darrell, “Playful agentic robot learning,” 2026. [Online]. Available: <https://arxiv.org/abs/2606.19419>
- [16] OpenAI, “GPT-6 Astra,” 2026, official model documentation, accessed September 15, 2026. [Online]. Available: <https://developers.openai.com/api/docs/models/gpt-6-astra>
- [17] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, “Scaling laws for neural language models,” *arXiv preprint arXiv:2001.08361*, 2020. [Online]. Available: <https://arxiv.org/abs/2001.08361>
- [18] J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, D. de Las Casas, L. A. Hendricks, J. Welbl, A. Clark, T. Hennigan, E. Noland, K. Millican, G. van den Driessche, B. Damoc, A. Guy, S. Osindero, K. Simonyan, E. Elsen, O. Vinyals, J. W. Rae, and L. Sifre, “Training compute-optimal large language models,” *arXiv preprint arXiv:2203.15556*, 2022. [Online]. Available: <https://arxiv.org/abs/2203.15556>
- [19] S. Sartor and N. Thompson, “Neural scaling laws in robotics,” 2024. [Online]. Available: <https://arxiv.org/abs/2405.14005>
- [20] F. Lin *et al.*, “Data scaling laws in imitation learning for robotic manipulation,” 2024. [Online]. Available: <https://arxiv.org/abs/2410.18647>
- [21] L. Zha *et al.*, “Guiding data collection via factored scaling curves,” 2025. [Online]. Available: <https://arxiv.org/abs/2505.07728>
- [22] G. Lu, W. Guo, C. Zhang, Y. Zhou, H. Jiang, Z. Gao, Y. Tang, and Z. Wang, “VLA-RL: Towards masterful and general robotic manipulation with scalable reinforcement learning,” 2025. [Online]. Available: <https://arxiv.org/abs/2505.18719>
- [23] Z.-h. Ding and Z. Wang, “SkillPlug: Unsupervised skill mining for few-shot adaptation in robotic manipulation,” 2026. [Online]. Available: <https://arxiv.org/abs/2607.08354>
- [24] C. Wang, C. Zhang, Z. Wu, R. Li, A. Ma, K. Chao, Y. Liang, X. Xu, Z. Wang, Y. Tang, and J. Lu, “SkillMemo: Expert-guided skill memory for compositional embodied manipulation,” 2026. [Online]. Available: <https://arxiv.org/abs/2608.05970>
- [25] A. Mandlekar *et al.*, “MimicGen: A data generation system for scalable robot learning using human demonstrations,” 2023. [Online]. Available: <https://arxiv.org/abs/2310.17596>
- [26] M. Xu, Z. Xu, C. Chi, M. Veloso, and S. Song, “XSkill: Cross embodiment skill discovery,” 2023. [Online]. Available: <https://arxiv.org/abs/2307.09955>
- [27] Y. Zhu, W. Wang, S. Wu, Y. Shi, and J. Wang, “Learning semantic atomic skills for multi-task robotic manipulation,” 2025. [Online]. Available: <https://arxiv.org/abs/2512.18368>
- [28] S. Zhang, S. Wang, H. Tang, J. Wu, and L. Nie, “Learning new tasks via reusable skills: Skill-compositional experts for embodied continual learning,” 2026. [Online]. Available: <https://arxiv.org/abs/2606.15685>
- [29] B. Quartey, S. Castro, E. Rosen, W. Thomason, G. Konidaris, and S. Tellex, “Jointly learning predicates and actions enables zero-shot skill composition,” 2026. [Online]. Available: <https://arxiv.org/abs/2605.20648>
- [30] M. Ahn *et al.*, “Do as i can, not as i say: Grounding language in robotic affordances,” 2022. [Online]. Available: <https://arxiv.org/abs/2204.01691>
- [31] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, and A. Zeng, “Code as policies: Language model programs for embodied control,” 2022. [Online]. Available: <https://arxiv.org/abs/2209.07753>
- [32] W. Huang, C. Wang, Y. Li, R. Zhang, and L. Fei-Fei, “ReKep: Spatio-temporal reasoning of relational keypoint constraints for robotic manipulation,” 2024. [Online]. Available: <https://arxiv.org/abs/2409.01652>
- [33] Y. Liu, W. Chen, X. Song, T. Pu, S. Mo, Y. Bai, Z. Chen, Q. Sun, L. Zhong, Y. Shen, and L. Lin, “PhyAgentOS: A self-evolving operating system for embodied agents with decoupled cognitive planning and physical execution,” 2026. [Online]. Available: <https://arxiv.org/abs/2607.16636>
- [34] X. Qin, S. Luan, J. See, C. Yang, and Z. Li, “AEROS: A single-agent operating architecture with embodied capability modules,” 2026, submitted to Engineering Applications of Artificial Intelligence. [Online]. Available: <https://arxiv.org/abs/2604.07039>
- [35] S. Yoon, M. Yoo, S. Ahn, S. Choi, and H. Woo, “RoboBRIDGE: A modular framework for bridging policies to robust real-world robotic agents,” 2026, accepted to IROS 2026. [Online]. Available: <https://arxiv.org/abs/2607.27881>
- [36] S. Wu, Y. Ji, Q. Li, Z. Zhang, Q. He, W. Xie, G. Zhang, B. Bayramli, Y. Ding, and H. Lu, “Dejavu: Towards experience feedback learning for embodied intelligence,” 2025. [Online]. Available: <https://arxiv.org/abs/2510.10181>
- [37] G. Tzifas and H. Kasaei, “Lifelong robot library learning: Bootstrapping composable and generalizable skills for embodied control with language models,” 2024, iCRA 2024. [Online]. Available: <https://arxiv.org/abs/2406.18746>
- [38] B. Liu *et al.*, “LIBERO: Benchmarking knowledge transfer for lifelong robot learning,” 2023. [Online]. Available: <https://arxiv.org/abs/2306.03310>
- [39] S. Fei, S. Wang, J. Shi, Z. Dai, J. Cai, P. Qian, L. Ji, X. He, S. Zhang, Z. Fei, J. Fu, J. Gong, and X. Qiu, “LIBERO-Plus: In-depth robustness analysis of vision-language-action models,” *arXiv preprint arXiv:2510.13626*, 2025. [Online]. Available: <https://arxiv.org/abs/2510.13626>
- [40] M. J. Kim, C. Finn, and P. Liang, “Fine-tuning vision-language-action models: Optimizing speed and success,” *arXiv preprint arXiv:2502.19645*, 2025. [Online]. Available: <https://arxiv.org/abs/2502.19645v2>
- [41] S. Fei, S. Wang, L. Ji, A. Li, S. Zhang, L. Liu, J. Hou, J. Gong, X. Zhao, and X. Qiu, “SRPO: Self-referential policy optimization for vision-language-action models,” *arXiv preprint arXiv:2511.15605*, 2025. [Online]. Available: <https://arxiv.org/abs/2511.15605v2>
- [42] X.-M. Wu, B. Fan, K. Liao, J.-j. Jiang, R. Yang, Y. Luo, Z. Wu, W.-S. Zheng, and C. C. Loy, “VLANeXt: Recipes for building strong VLA models,” *arXiv preprint arXiv:2602.18532*, 2026. [Online]. Available: <https://arxiv.org/abs/2602.18532v2>
- [43] InternRobotics, “InternVLA-A1.5-Libero: Model card and benchmark results,” 2026, accessed September 15, 2026. [Online]. Available: <https://huggingface.co/InternRobotics/InternVLA-A1.5-Libero>
- [44] H. Chen, J. Liu, Z. Yan, N. Han, R. Zhang, C. Gu, J. Gao, Z. Guo, S. Qian, Y. Wang, P. Jia, S. Zhang, and P.-A. Heng,

- “LaST-R1: Reinforcing robotic manipulation via adaptive physical latent reasoning,” *arXiv preprint arXiv:2604.28192*, 2026. [Online]. Available: <https://arxiv.org/abs/2604.28192v2>
- [45] Z. Zhan, Y. Chen, J. Zhou, Q. Lyu, H. Liu, K. Wang, L. Lin, and G. Wang, “Stable language guidance for vision-language-action models,” in *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2026, pp. 4137–4159. [Online]. Available: <https://aclanthology.org/2026.acl-long.190/>
- [46] C. Xu, Y. Xu, M. Xue, J. Lin, J. Wang, X. Zhang, Y. Wang, and J. Yu, “The curse of precision: A data scaling law for high-precision robotic manipulation,” 2026, accepted to ICRA 2026. [Online]. Available: <https://arxiv.org/abs/2607.23108>
- [47] W. Huang, C. Wang, R. Zhang, Y. Li, J. Wu, and L. Fei-Fei, “VoxPoser: Composable 3d value maps for robotic manipulation with language models,” 2023. [Online]. Available: <https://arxiv.org/abs/2307.05973>
- [48] W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence, A. Zeng, J. Tompson, I. Mordatch, Y. Chebotar, P. Sermanet, N. Brown, T. Jackson, L. Luu, S. Levine, K. Hausman, and B. Ichter, “Inner monologue: Embodied reasoning through planning with language models,” 2022. [Online]. Available: <https://arxiv.org/abs/2207.05608>
- [49] B. Ai *et al.*, “Towards embodiment scaling laws in robot locomotion,” 2025. [Online]. Available: <https://arxiv.org/abs/2505.05753>
- [50] T. Pearce *et al.*, “Scaling laws for pre-training agents and world models,” 2024. [Online]. Available: <https://arxiv.org/abs/2411.04434>
- [51] K. Zhu *et al.*, “Scaling test-time compute for LLM agents,” 2025. [Online]. Available: <https://arxiv.org/abs/2506.12928>
- [52] J. Kwok, C. Agia, R. Sinha, M. Foutter, S. Li, I. Stoica, A. Mirhoseini, and M. Pavone, “RoboMonkey: Scaling test-time sampling and verification for vision-language-action models,” 2025. [Online]. Available: <https://arxiv.org/abs/2506.17811>
- [53] Z. Zhao, M. Cho, H. Shen, B. Zheng, K. Gao, Y. Cao, and Z. M. Mao, “Test-time scaling for world action models via zero-shot geometric evaluation,” 2026, extended version of a CVPR 2026 EAI workshop paper. [Online]. Available: <https://arxiv.org/abs/2607.17454>
- [54] S. Nasiriany *et al.*, “RoboCasa: Large-scale simulation of everyday tasks for generalist robots,” 2024. [Online]. Available: <https://arxiv.org/abs/2406.02523>
- [55] Z. Wang, B. He, K. Yu, S. Lee, R. Gao, F. Huang, and Y. Aloimonos, “HumanEgo: Zero-shot robot learning from minutes of human egocentric videos,” 2026. [Online]. Available: <https://arxiv.org/abs/2605.24934>
- [56] C. Tie, S. Sun, J. Zhu, Y. Liu, J. Guo, Y. Hu, H. Chen, J. Chen, R. Wu, and L. Shao, “Manual2Skill: Learning to read manuals and acquire robotic skills for furniture assembly using vision-language models,” 2025. [Online]. Available: <https://arxiv.org/abs/2502.10090>

APPENDIX A

WHY END-TO-END SCALING REMAINS COSTLY

A. Collecting Physical Interaction Data

Large language models began with an unusual historical advantage: vast quantities of text, code, books, and webpages had already been produced for human purposes. Model builders still pay heavily to curate and train on that material, but they do not need to recreate every sentence by operating a physical machine. General robot learning faces a different starting condition. Much of its valuable evidence consists of synchronized observations, actions, embodiment state, contact, calibration, failure, recovery, and final physical effect. This evidence does not exist until a robot or simulator is run. Open X-Embodiment, DROID, RoboMIND, and AgiBot World are major advances [7], [8], [9], [10]; their scale also makes visible how much new infrastructure and interaction must be created before robot data can resemble a mature web-scale substrate.

The most consequential evidence is often the least likely to appear in a successful demonstration set: near collisions, transparent or deformable objects, unstable grasps, partial insertion, sensor disagreement, recovery after an interrupted sequence, and the chained states produced by earlier mistakes. These events determine whether a robot is useful outside a

curated distribution. A broad visual backbone can contribute language, object semantics, and strong priors, yet it does not automatically supply embodiment-specific geometry, contact dynamics, or action-conditioned consequences [11]. Robot generality therefore depends on both model capacity and the cost of collecting representative physical interaction.

B. Coverage Grows by Interacting Factors

The difficulty is not captured by a single count of demonstrations. Let a deployment domain contain F physically relevant factors, including object identity, pose, clutter, illumination, camera, gripper, material, contact mode, and task relation. If n_j denotes the meaningfully different regimes for factor j , a dense Cartesian design contains

$$N_{\text{dense}} = \prod_{j=1}^F n_j \quad (11)$$

coverage cells, where $n_j \geq 1$. This idealized product shows why physical coverage can grow much faster than any one dataset coordinate: adding another independent factor multiplies the joint space. Even pairwise screening must consider $F(F-1)/2$ factor pairs, with $\sum_{j < k} n_j n_k$ joint pairwise cells. Factored scaling curves were introduced precisely because exhaustive environmental variation is prohibitively expensive [21].

Precision creates a second pressure. Let P denote a target tolerance, where a smaller value is stricter and c is the smallest attainable tolerance. One recent analysis reports a regime of the form $\log N_{\text{demo}} \propto 1/(P-c)$ for $P > c$ as $P \downarrow c$ [46]. This relation shows why producing the final increments of physical precision can require sharply more data.

C. The Retraining Tax Is Global Even When Failure Is Local

Suppose a policy fails only for one unfamiliar container under one camera and grasp configuration. The missing competence is local, but repairing a monolithic policy commonly reopens a global process: collect representative interaction, optimize a new version, and recheck previously supported behavior. Adapters, replay, modular heads, online reinforcement learning, and continual-learning methods can reduce this burden. They do not remove the systems fact that a change distributed through model parameters is difficult to inspect, delimit, and certify locally. As the policy’s coverage expands, the cost of deciding what else may have changed expands with it.

The *retraining tax* arises when a local gap requires updating and revalidating the whole policy. Teach-and-Grow changes the unit of update: a missing behavior can be added as a separately addressable Skill Block, tested where it applies, and linked to existing blocks. Appendix G develops the scaling and cost consequences of this change.

D. The Modality and Embodiment Tax

Adapting a VLA to touch, audio, force, point clouds, or a new camera requires an observation interface and aligned interaction data. A new gripper or robot likewise requires an appropriate action interface, calibration, and validation. In TGL, new sensing enters the blocks that consume its evidence, and

a new embodiment is supported through compatible executors and grounding. Integration is tested at these interfaces and their affected compositions.

E. The Long Tail Is Where Explicit Knowledge Matters Most

End-to-end policies are strongest when deployment resembles their training data. Long-tail failures are different: they are sparse, varied, and often useful only after diagnosis. A failed grasp matters more when the robot records that a transparent wall confused depth, a side approach caused collision, or retention must be checked before transport. Experience Memory keeps these lessons; Skill Blocks keep the response. When retrieval, grounding, or control fails, the robot can point to what it believed, what it expected, and what local object should change.

APPENDIX B

TEACH-AND-GROW LEARNING VERSUS EXISTING LEARNING FORMS

A. The Learned Object and the Update Operator

Table IV compares the learned objects and update mechanisms. In TGL, demonstrations define an initial hypothesis about semantic effects, ordering, and reusable relations. Current objects, geometry, grasps, paths, and commands remain runtime variables. The robot can realize a taught structure through different backends, acquire a recovery absent from the demonstrations, and update its explicit behavior state while the foundation policy remains fixed.

The difference can be expressed through the update operator. Parametric learning changes a model,

$$\theta_{n+1} = \text{Optimize}(\theta_n, \mathcal{D}_n), \quad (12)$$

whereas Teach-and-Grow applies Eq. 3 to the explicit skill, memory, and history state while the foundation parameters θ stay fixed. The two operations are compatible. A learned policy may implement a block, and verified behavior may later be distilled into separate student parameters ϕ . Explicit acquisition and parametric training can therefore operate at different stages of the same learning system.

B. Agent Decisions over the Evolving Task Plan

Language and vision-language models already organize robot behavior as skill selectors, program generators, spatial reasoners, and constraint builders. SayCan ranks available skills, Code as Policies writes programs against robot APIs, VoxPoser and ReKep convert language and images into spatial objectives or constraints, and Inner Monologue incorporates environmental feedback into planning [30], [31], [47], [32], [48]. These systems provide the planning, tool-use, and feedback mechanisms on which later agentic architectures build.

In TGL, the agent decides how the task plan should evolve during execution. It chooses evidence, blocks, and tools; checks whether the last action achieved its effect; and revises the remaining plan. It also selects experience to retain and candidate behaviors to validate. A coding agent uses a workspace in a

similar way, repeatedly inspecting state, acting through tools, and updating the plan and stored artifacts. For the robot, observations describe the physical scene, tools execute actions, and persistent artifacts contain reusable skills and experience.

Recent systems such as LRLL, ASPIRE, SkillMemo, SCE, and PACTS study lifelong skill acquisition, agentic discovery, memory, and compositional reuse [37], [14], [24], [28], [29]. TGL belongs to this family. It connects few-shot abstraction, closed-loop execution, and persistent learning: teaching provides a subgoal strategy, execution tests its composition, and the resulting experience updates the Skill Library and Experience Memory. Mature behavior can later supply data for fast-policy distillation.

C. Teaching and Learning Promote One Another

The phrase 教学相长 comes from the classical Chinese educational text *Xue Ji* and is commonly translated as *teaching and learning promote one another*. In TGL, the teacher demonstrates a useful task structure. Execution then reveals what needs clarification: an object role, a recovery, an embodiment-specific requirement, or an outcome test. That evidence informs the next teaching intervention.

The resulting loop is

$$\text{teach}_n \rightarrow \text{compose}_n \rightarrow \text{act+check}_n \rightarrow \text{grow}_n \rightarrow \text{teach}_{n+1}. \quad (13)$$

Early teaching may introduce entire behaviors. Later teaching can focus on the frontier: rare failures, new tools, or missing transitions between familiar blocks. Each attempt reveals what the robot can do and where further guidance would help. This mutual refinement is the educational principle behind the architecture and the practical basis for the prediction that teaching demand should fall as reusable experience accumulates.

APPENDIX C

FROM ZERO-SHOT AGENT CONTROL TO TEACH AND GROW

A. The Minimal Agent Route

The project began by asking a multimodal Agent to plan directly from robot-view images. It understood the goal, but understanding the task was not enough to execute it: depth, collision, contact, and high-rate control still demanded robot-native competence.

B. Tool Use and Agentic Visual Feedback

We then gave the Agent perception, grasping, planning, and control tools, and returned a fresh observation after each short execution. It could now change its route when the world disagreed with the plan. This made zero-shot agent control technically feasible, but learning a physical task could still require many trials and model calls.

Structured memory made those trials useful beyond the current episode: successes supplied strategies, while failures supplied conditions and repairs. The update changed explicit memory rather than model parameters.

TABLE IV
DIFFERENT LEARNING OBJECTS AND UPDATE MECHANISMS.

Property	Behavior cloning / imitation	Reinforcement learning	Teach-and-Grow Learning
Primary evidence	Expert observation–action pairs	Interaction, reward, transitions	Demonstrations, instructions, success, failure, diagnosis, recovery
Learned object	Parametric policy	Policy, value function, or world model	Explicit Skill Library and task compositions
Demonstration role	Target behavior to imitate	Initialization, prior, or guidance	Seed evidence for semantic structure and reusable strategy
Exploration	Usually limited	Central	Targeted after few-shot teaching; bounded by safety and missing knowledge
Failure signal	Distribution mismatch or supervised error	Reward / return	Structured cause, negative condition, recovery, or missing block
Update	Gradient optimization	Gradient or search-based optimization	Create, narrow, merge, replace, compose, or retire Skill Blocks
Persistence	Distributed in parameters	Distributed in parameters / replay	Explicit, versioned, human-readable behavior objects

C. Why Few-Shot Teaching Is the Practical Default

Unrestricted exploration is slow and can be unsafe on a physical robot. Few-shot teaching provides a successful starting structure, so autonomous behavior can focus on the variation, correction, and recovery that the demonstrations did not cover. This became the default Teach-and-Grow route.

Multiple sequential model calls make the current Agent more expensive to run than feed-forward policy inference. This motivates the proposed slow-teacher/fast-student design: use the Agent to acquire unfamiliar behavior and produce structured trajectories, then train fast policies for mature tasks. Improvements in model efficiency and caching may also reduce inference cost.

- **articulation and activation blocks:** open, close, rotate, press, toggle, or actuate;
- **observation blocks:** gather evidence needed by later decisions;
- **recovery blocks:** restore a precondition after a diagnosed failure.

These families organize retrieval by intended effect. Each block can use a learned policy, planner, servo, or human intervention suited to that effect.

C. A Populated Block from the Experiment

The second induced block represents *release the held object through the visible opening*. It retains the relation and release strategy; the opening frame, release band, held-object offset, path, and controller commands are computed in the current scene.

APPENDIX D SKILL BLOCKS AND ATOMIC SKILLS

A. Why the Term Skill Block

Atomic skills are an established and useful concept. The word *atomic*, however, often suggests the smallest indivisible controller, action primitive, or short policy. Our decomposition operates at a broader semantic level. A block is one decision-level building element of a task and may contain a substantial internal feedback loop. The block metaphor also makes the scaling argument intuitive: new tasks are built by recombining old blocks and learning only the missing pieces.

A short gripper close can be a block when it is independently meaningful. So can a longer *acquire the requested container* behavior that detects, approaches, closes, lifts, and checks retention. The granularity is determined by semantic closure and reuse, not by duration or number of motor commands.

B. Taxonomy

Skill Blocks can be organized by the kind of effect they produce:

- **acquisition blocks:** establish stable possession or contact;
- **transport blocks:** preserve a state while changing a spatial relation;
- **release and placement blocks:** establish support, containment, alignment, or insertion;

APPENDIX E DETAILED AGENT-LED SKILL INDUCTION

A. Source Normalization

Teaching is first converted into observable event sequences. Robot demonstrations provide synchronized observations and actions; human video provides visible state changes and object relations; written instructions provide intent and order. The source type stays attached because each source supports a different part of a block.

B. Semantic Decomposition

The Agent splits each trajectory at meaningful state changes using task language, object relations, gripper state, and motion. A segment receives the semantic signature

$$\sigma(s) = \langle g_s, u_s, a_s, \ell_s, p_s, q_s \rangle, \quad (14)$$

where g_s is the subgoal, u_s the manipulated entity, a_s its affordance, ℓ_s the target relation, p_s the preceding effect, and q_s the ordering context.

TABLE V
A POPULATED ACQUISITION SKILL BLOCK INDUCED FROM THREE TEACHER TRAJECTORIES.

Field	Stored content
Subgoal g	Acquire and retain the requested alphabet-soup can.
Scope $\mathcal{S}$	Object-specific in the reviewed LIBERO-Object task; it is not exposed as a generic can skill.
Reusable strategy ρ	Bind the current object frame; approach the visible graspable body; align; close with bounded local motion; verify that the object is held; lift while preserving retention.
Grounding $\gamma_i(o_t)$	Estimate current object identity, frame, confidence, obstacles, reachability, grasp candidates, and local motion parameters from fresh observations.
Executors Π_i	Current-scene grasp selection, servo acquisition, and a newly planned collision-aware transition.
Outcome test v_i	Pass only when fresh observations and action feedback verify object retention and displacement.
Recovery $\mathcal{R}$	Reobserve, adjust local parameters, invoke an allowed grasp repair, choose an alternative block, recompose the task, or abort.
Excluded	Demonstration world coordinates, pixels, old path, joint trajectory, and low-level action replay.

C. Cross-Demonstration and Cross-Embodiment Alignment

Alignment follows semantic effect rather than motor coordinates. *Acquire the requested container* can preserve the object role and the result *object is stably held* whether a robot uses a parallel gripper, suction, or a dexterous hand. The receiving robot supplies its own executor, grounding, and local verification. This defines a possible route to cross-embodiment reuse. The present study uses one embodiment.

The alignment procedure follows four steps:

- 1) normalize each source into observable events while keeping its type;
- 2) align segments that achieve the same effect and have compatible entities or relations;
- 3) separate shared task structure from pose, path, timing, and embodiment-specific control;
- 4) validate the resulting block on fresh cases and narrow its scope when needed.

Variable-length demonstrations therefore need not have the same number of raw segments. They must agree on the semantic effects used for the final task composition.

D. Invariant Synthesis

For each aligned group, the Agent separates:

- *semantic invariants*: subgoal, object role, relation, and expected effect;
- *behavioral invariants*: approach family, contact region, ordering, and permitted executor family;
- *instance variables*: pose, pixels, point cloud, grasp, path, timing, and robot configuration.

The first two categories define reusable strategy; instance variables are recomputed at runtime. A segment that still depends on the original coordinates remains an episode rather than a block.

E. Scope Selection

Scope expands only with supporting variation. Repeated success across poses supports pose invariance. Multiple instances support object-level abstraction. Cross-category success with a common affordance may support affordance-level reuse. Contradictory demonstrations create alternative blocks or narrower conditions. A single unusual instrument remains object-specific until evidence justifies more.

F. Validation and Library Update

A candidate block is tested on separated cases before normal retrieval. The test asks whether it produces the intended effect from the current scene, preserves matched old behavior, and stays within its claimed scope. A weak block is narrowed or repaired; a validated one enters a new library version.

APPENDIX F

AGENTIC EXECUTION, MEMORY, AND FAILURE LEARNING

A. The Agent's Task-Level Decisions

At task time, the agent decides:

- which observations and modalities are needed;
- which Skill Blocks are relevant;
- how they should be ordered or conditioned;
- which backend should instantiate a block;
- whether an observed effect is sufficient to continue;
- whether to retry, substitute, recompose, request teaching, or stop;
- which parts of an execution are valuable for future learning.

The agent makes these decisions at meaningful state transitions, while the block's executor handles continuous control.

B. Success and Failure as Memory

A successful trace can establish a new block, broaden evidence, or confirm a composition. A failure can be more informative than another success. It may reveal that a visual condition is ambiguous, a grasp family is unsuitable, a planner bridge fails under clutter, or a block's outcome test is too weak. The memory update records a structured explanation and links it to observations and outcomes.

Because the memory is explicit, humans can inspect and edit it. An operator can correct an overgeneralized rule, forbid a recovery, or mark a tool version incompatible. This editability is difficult to achieve when every experience is absorbed only into distributed parameters.

C. Four Levels of Retention

Persistent storage avoids direct parametric overwriting, yet behavior can still degrade. We distinguish:

- 1) **storage retention**: the block remains present;
- 2) **retrieval retention**: the correct block is still selected;
- 3) **grounding retention**: its strategy still binds to current sensing and embodiment;
- 4) **behavioral retention**: execution success is maintained.

A longitudinal evaluation measures all four to identify where a retained behavior stops being useful.

APPENDIX G

SCALING LAWS, COST, AND THE TEACH-AND-GROW HYPOTHESIS

A. What a Scaling Law Means

The influential language-model scaling laws did more than state that larger models perform better. They identified explicit resources such as model parameters, training tokens, and compute, then showed that held-out loss follows approximately predictable power-law trends over broad ranges [17]. Chinchilla further showed that the allocation of a fixed compute budget matters: model size and data should grow together near the compute-optimal frontier [18]. A useful scaling law needs four things: a measurable resource, an outcome that matters, a stable relationship between them, and a prediction that holds at scales not used to fit the curve.

For robots, the scaling relation must specify which experience counts, what remains fixed, what outcome is predicted, and where the relationship should saturate or fail.

B. What Existing Robot Scaling Laws Measure

Robot learning already exhibits several valuable scaling regularities. A broad meta-analysis finds power-law-like improvement with data, model size, and compute across many robot-learning studies [19]. Large imitation-learning experiments report approximately power-law gains with demonstrations, environments, and object diversity, with diversity often more valuable than repeatedly sampling the same condition [20]. Factored scaling curves estimate which environmental factors most deserve additional data [21]; embodiment scaling studies ask how performance changes as the diversity of bodies grows [49]; and agent/world-model work studies pretraining resources and architecture-dependent scaling [50]. A parallel line spends more computation at inference through sampling, verification, or world-action evaluation [51], [52], [53].

These laws answer important questions, but they mostly concern one of three regimes: how a policy improves before deployment as offline resources grow; how a family of policies changes with training diversity; or how one episode improves as test-time computation increases. They do not directly describe a deployed robot that repeatedly turns new experience into persistent, separately reusable competence and then faces another task with a different acquisition burden.

C. Why Performance Scaling and Cost Scaling Diverge

A benign performance curve can hide a steep data-production curve. Suppose a manipulation domain is broadened along F interacting factors, each with roughly r regimes. Dense coverage scales as r^F in the idealized Cartesian case of Eq. 11.

A learner may generalize between cells and need far fewer demonstrations, but newly discovered interactions still require fresh robot operation. Near a tolerance limit, $\log N_{\text{demo}} \propto 1/(P - c)$ with $P > c$ predicts sharply increasing demand as $P \downarrow c$ [46].

For an end-to-end route, a useful cost decomposition is

$$C_{\text{E2E}}(K) = C_{\text{collect}}(N_{\text{cover}}(K)) + C_{\text{train}}(N_{\text{cover}}(K)) + C_{\text{regress}}(\theta_{\text{new}}(K), \mathcal{T}_{\text{old}}), \quad (15)$$

Here K counts matched semantic capability additions, $N_{\text{cover}}(K)$ is the interaction needed to cover them, $\theta_{\text{new}}(K)$ is the resulting updated policy, and $\mathcal{T}_{\text{old}}$ is the earlier task set. All cost terms in this section use one preregistered scalar unit, so collection, compute, human time, validation, and regression are converted before they are added. Equation 15 identifies the regime in which dense joint coverage, strict tolerance, and repeated global regression make the next useful increment increasingly costly.

Teach-and-Grow is designed around a different cost structure. In TGL, each counted semantic capability addition is one admitted reusable Skill Block. For block k , define

$$\kappa_k = \kappa_k^{\text{teach}} + \kappa_k^{\text{ground}} + \kappa_k^{\text{validate}} + \kappa_k^{\text{link}}, \quad (16)$$

$$C_{\text{TGL}}(K) = C_0 + \sum_{k=1}^K \kappa_k + C_{\text{retrieve}}^{\text{cum}}(K).$$

Here C_0 is the fixed infrastructure cost, and $C_{\text{retrieve}}^{\text{cum}}(K)$ is the retrieval cost accumulated while admitting the first K semantic capability additions, each realized by one block in TGL. If block scope and validation stay local, each κ_k remains bounded. With hierarchical retrieval satisfying $C_{\text{retrieve}}^{\text{cum}}(K) = O(K)$, cumulative cost is at most linear in K . The robot then pays mainly for the missing behavior and its local interfaces. All-to-all compatibility testing would instead make linking quadratic; block contracts and scopes are what keep the update local.

Fig. 5 contrasts the two acquisition regimes as a conceptual prediction: broadly coupled updates grow convexly, while local additions can remain near-additive.

D. The New Scaling Resource: Effective Reusable Experience

The core Teach-and-Grow law concerns benefit rather than storage volume. Equation 7 gives each stored experience a score in $[0, 1]$ based on five practical factors: evidence reliability, added coverage, retrievability, grounding validity, and compatibility with admitted blocks. The factors and aggregation rule are fixed before the scaling study, and future-task outcomes are hidden from the score.

With the foundation agent, tool versions, test-time budget, future-task distribution, and scoring rule fixed, Eq. 1 predicts that future-task error and teaching demand decay toward irreducible floors as power laws in X_n . Equation 16 adds the cost-side prediction: explicit capability can grow close to additively when updates remain local. The resource grows after deployment, and one task can make the next task easier without retraining the full policy.

CAPABILITY ACQUISITION · TWO COST REGIMES

Conceptual comparison under stated conditions

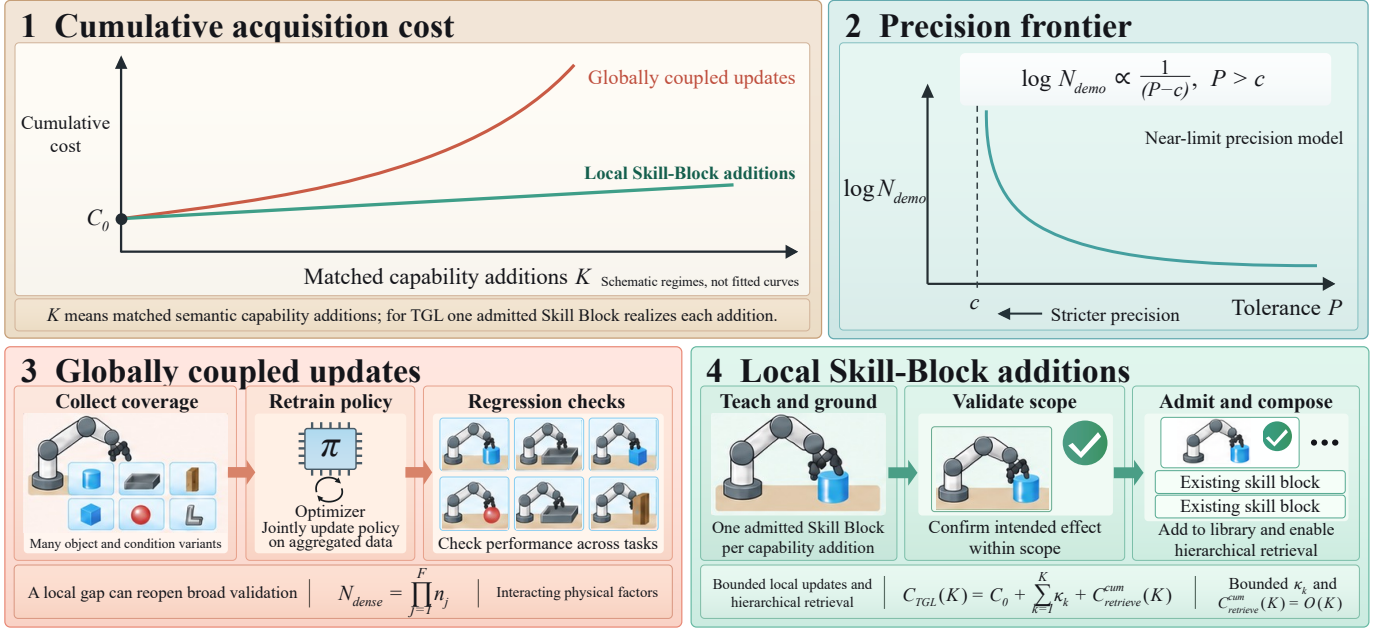


Fig. 5. Conceptual acquisition-cost regimes. When each addition reopens joint coverage and regression, globally coupled updates can grow convexly with capability breadth. Local Skill-Block additions can remain near-additive when interfaces stay bounded and retrieval is at most linear. Inset: for target tolerance $P > c$, the cited precision law diverges as $P \downarrow c$.

E. A Longitudinal Test

A scaling study freezes the foundation agent, tools, inference budget, success criteria, future-task distribution, and scoring rule while the robot learns tasks in sequence. Each library checkpoint receives an X_n value before the next outcomes are observed. The evaluation mixes old tasks, new compositions, new instances of known affordances, and tasks that require a genuinely new behavior. These groups show whether progress comes from remembering an episode, recombining a known skill, broadening its scope, or acquiring a missing block.

At every checkpoint, the primary variables are future-task error $\mathcal{E}_{future}(X_n)$ and teacher intervention time $D_{teach}(X_n)$. The latter runs until the fixed success rule or the preregistered teaching-budget cap; budget-exhausted acquisitions remain in the analysis at that cap. Retention, reuse, harmful retrieval, latency, and cumulative acquisition cost explain why a checkpoint departs from the main curve. Raw episodic memory, a fixed tool pipeline, sequential VLA adaptation, and an Agent-VLA hybrid provide matched points of comparison. The power law is fitted on early checkpoints and tested on larger held-out checkpoints alongside standard alternative curves. Total cost includes teaching, robot interaction, inference, verification, storage, retraining, and regression. The prediction is useful only if it reaches unseen checkpoints and reveals where improvement saturates.

APPENDIX H A WHOLE-SYSTEM VIEW OF THE ROBOT LEARNING ECOSYSTEM

A. Sources of Teaching and Experience

The outer network in Fig. 6 accepts more than robot demonstrations. Teleoperation and robot trajectories provide the richest embodiment-aligned evidence and can seed both blocks and fast policies [8]. Simulation, procedural generation, and sim-to-real pipelines can produce controlled variations, failures, and rare conditions before physical deployment [25], [54]. Human demonstration and egocentric video provide inexpensive semantic and interaction structure, although embodiment conversion must recover robot-executable geometry and action [26], [55]. Manuals, natural-language instructions, diagrams, and CAD can provide ordering, constraints, affordances, and object-specific procedures [56]. Autonomous exploration contributes the complementary evidence that teachers rarely provide: failures, repairs, and unexpected but successful alternatives.

These sources differ in what they can justify. A manual may reveal that a latch must be released before a panel moves, but it cannot by itself certify a grasp controller. Human video may reveal contact sequence and object relation while leaving robot kinematics unresolved. Simulation may offer exact state and broad variation yet still require real-world calibration. TGL preserves source type and uncertainty so that semantic structure can transfer while physical claims are validated by an appropriate executor and observation channel.

TGL · A CONNECTED ROBOT LEARNING ECOSYSTEM · 教学相长

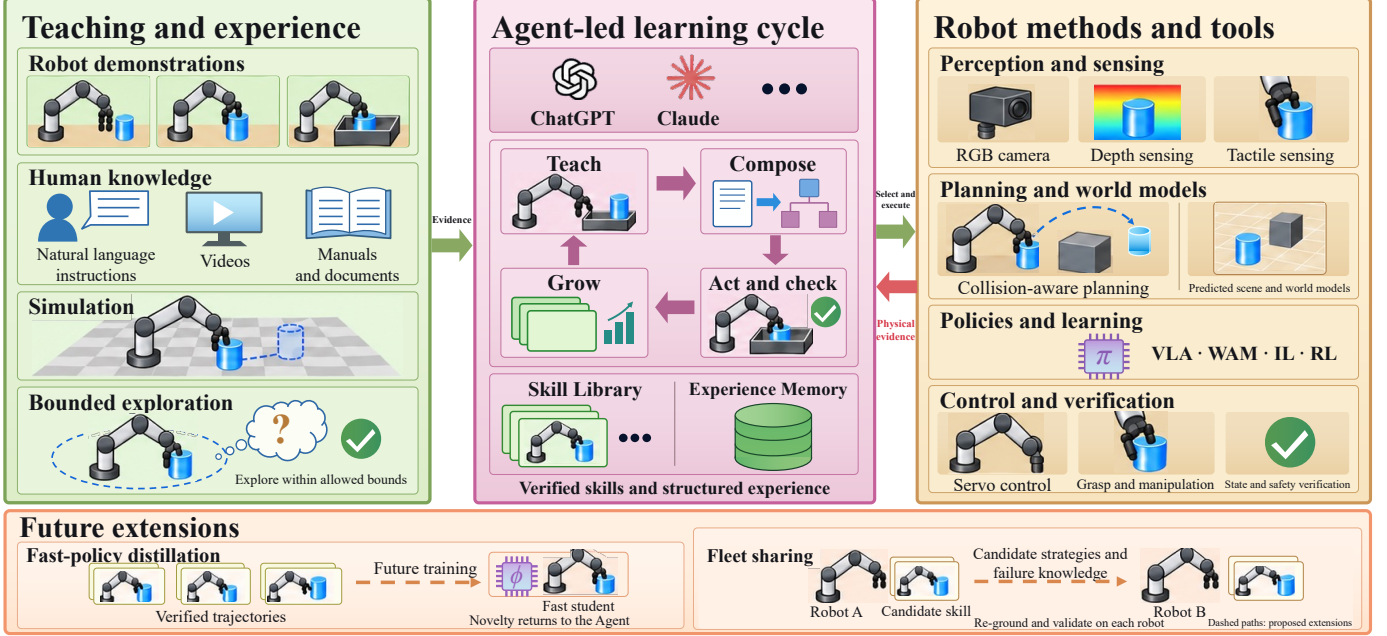


Fig. 6. Whole-system ecology of Teach-and-Grow Learning. TGL provides a semantic routing and verification layer through which human knowledge, simulation, learned policies, learning methods, perception, planning, control, and safety mechanisms contribute evidence or executable change. The Agent composes these specialized roles around a common learning cycle; verified capability and experience persist separately for later tasks. Dashed paths denote future distillation and fleet sharing.

B. Roles of Learned Policies and Classical Methods

A VLA, WAM, diffusion policy, or task-specific policy can occupy three roles in the architecture. It may implement a Skill Block, execute a mature composition as a fast path, or serve as a future student of verified agentic trajectories. Imitation learning and reinforcement learning can build or improve these executors. The surrounding Teach-and-Grow system selects an applicable policy, supplies the evidence it needs, checks its effect, and chooses the next step when execution leaves its supported range.

Classical robotics is equally native to the system. Detection, segmentation, RGB-D geometry, grasp synthesis, motion planning, impedance control, visual servoing, and tactile feedback can each provide evidence or realize a block. Traditional controllers are often the most reliable backend for a well-understood local effect. World models can predict consequences or rank alternatives; test-time verifiers can reject weak candidates; safety filters and humans can constrain the action set. The top-level task is no longer forced into a universal perception–planning–control pipeline. Each block assembles the physical loop appropriate to its own semantic effect.

C. Coordinating Models and Tools through Skill Blocks

The common interface is a semantic contract: what effect a block is intended to establish, where it applies, what current evidence it requires, which backends can realize it, how success is observed, and what recovery is available. The agent uses those contracts to connect methods that were previously evaluated as separate systems. A visual servo can follow a VLA

proposal; a planner can bridge two learned policies; a world model can screen candidate blocks; a human can teach the one transition that simulation failed to discover. The contract allows both classical and learned components to contribute evidence or execute an action within the same block.

Integration remains local: a new module needs its own adapter, calibration, and validation, while unrelated blocks remain unchanged. A tactile sensor can strengthen the blocks that need contact, and a new VLA can replace one executor without erasing a proven visual-servo block. The system can adopt stronger components while preserving the learning cycle around them.

D. Verified Data Generation and Distillation

A slow agentic learner produces richer supervision than a raw successful trajectory. Its records can include semantic boundaries, selected blocks, current bindings, outcomes, verifier decisions, failure classes, recoveries, and final task success. These signals can train a student to execute one block, choose among blocks, or directly map observations to actions for a mature task family. A practical robot can alternate between acquisition and compression:

novel task → few-shot agent solution → verified experience
→ distilled fast student → routine deployment. (17)

Uncertainty, distribution shift, or failure returns control to the agentic route, which expands both the library and the next student dataset. The slow system learns at the frontier; the fast system serves conditions that have become familiar.

E. Personal, Fleet, and Ecosystem Growth

At the personal level, one robot accumulates skills adapted to its own tools, calibration, and environment. At the fleet level, robots can share candidate semantic strategies, failure signatures, outcome tests, and distilled students; each receiving robot re-grounds and revalidates them with its own embodiment. At the ecosystem level, third parties may publish interaction blocks, specialist perception modules, VLA executors, recovery methods, embodiment adapters, simulators, and validation suites.

Teach-and-Grow connects these contributions within one learning process. Foundation models supply priors; demonstrations, simulation, and human knowledge supply teaching; learned and classical methods supply physical control. The Agent coordinates them, while persistent skills and memory retain useful behavior and context. Future distillation can convert mature behavior into faster execution.

APPENDIX I EXPERIMENTAL DETAILS

A. Agent Configuration and Trace Provenance

The benchmark implementation uses OpenAI GPT-6 Astra [16], whose official model identifier is `gpt-6-astra`, with Codex as the agent interface. The two earlier online traces analyzed below used `gpt-5.6-sol`, seed 0, and structured decisions. In those traces, the Agent received the task, current agent-view and wrist evidence, registered Skill Blocks, allowed tool routes, and verifier or planner evidence. Each trace allowed at most one evidence refresh and two replans; no human changed an in-episode decision.

The two representative successful traces come from a frozen LIBERO-GOAL archive. Together they show the two decisions studied here: rebuilding the route for bowl-on-plate and gathering a new observation for drawer opening.

B. Visual Demonstration Decomposition

The visual study uses ten successful demonstrations from two related LIBERO-Object condiment-transfer tasks, five per task. Sampled agent-view and wrist images are processed without reward, task-success labels, hidden simulator object state, demonstration actions, or reference boundaries as production inputs. Independent annotations are used only after prediction. The corpus contains 40 predicted and 40 reference stages.

Ordered role/type accuracy is 1.000. Boundary F1 is 0.100 at exact matching, 0.633 within one sampled frame, and 0.900 within two sampled frames. Observable tests verify ten acquisition and ten release effects, producing 20/20 effect confirmations.

C. Task-Specific Skill Learning

Three teacher trajectories from states 0–2 produce an object-specific acquisition block and a relation-aware release block. On separated states 3–5, the pair succeeds on 3/3 evaluation initial states; the existing six-block route also succeeds on 3/3. After scope checks, the two blocks enter an eight-block library, survive save-and-reload, and again succeed on 3/3. On farther states 6–8, the learned route stops at the missing semantic effect, showing that its scope is enforced during execution.

D. Related-Task Fixed-Executor Pilot

The pilot compares six-block and eight-block libraries on two related LIBERO-Object tasks. Executor, runtime, seed 0, success criteria, and the three-attempt budget stay fixed; only the learned acquisition and release blocks are added. Library contents and scopes are frozen before evaluation. Onboarding uses states 12–14, and both libraries are evaluated on the same disjoint states 15–17.

With unsuccessful acquisitions retained at the three-attempt cap, the observed median is 2.5 attempts with six blocks and 1.5 with eight. Evaluation success is 0/6 versus 4/6; the corresponding 95% Wilson intervals are $[0.00, 0.39]$ and $[0.30, 0.90]$. Both libraries use the same two tasks, seed, initial states, and execution budget.

E. Failure-Localization Cohort

A separate eight-attempt acquisition cohort records 0/8 task success and localizes the remaining failures: two occur before motion planning, four arise from path consistency or calibration, and two from gripper closure. These categories identify where the feedback loop should improve next.

APPENDIX J CURRENT SYSTEM

The GPT-6 Astra/Codex implementation selects blocks and tools, checks outcomes, and revises plans using task instructions and current visual and execution evidence. Detection, segmentation, RGB-D, and robot state ground actions. Contact-GraspNet and MPLib supply grasps and collision-aware motion; controllers execute actions, and observations verify effects. The Skill Library and Experience Memory retain behaviors, scopes, compositions, and context.